MODULE 21 · DOMAIN 3 - DEPLOYMENT, PROVISIONING, AND AUTOMATION

Images and Containers for Operators

EC2 Image Builder, AMI governance, ECR, ECS and EKS - all newly in scope for SOA-C03.

DOMAIN 3 of 5 EXAM WEIGHT 22% MODULE 21 of 35

admin@sthithapragna.com

An image is a **deployment artefact**. The operator's job is its lifecycle - built consistently, scanned, distributed, governed and expired - not what is inside it.

The golden AMI pattern

- Bake packages, agents, hardening and application code **into the image** rather than installing them at launch.
- Launch time drops from minutes to seconds, which module twelve showed is the difference between winning and losing a scaling race.
- Every instance is **identical**, so drift between instances stops being possible.
- The cost is a **rebuild cadence**. An image that is never rebuilt is a security debt that grows every month.

> ON THE EXAM
>
> Module twelve's warm pool decision reappears here. Installation belongs in the image; loading current data does not.

EC2 Image Builder: the four pieces

- **Recipe**: a parent image plus ordered **components** and volume configuration. Image recipes make AMIs; container recipes make container images.
- **Components** are YAML documents with **build** and **test** phases. AWS provides many; you write the rest.
- **Infrastructure configuration**: the instance type, subnet, IAM role and logging used **during the build**.
- **Distribution configuration**: which Regions and accounts receive the finished image, and its launch permissions.

> ON THE EXAM
>
> The test phase is what makes this more than a script. An image that fails its tests is never distributed.

An Image Builder pipeline

01
Trigger
A schedule, a dependency update, or a manual run.

02
Build
Launch a temporary instance, apply components in order.

03
Test
Run test components. Failure stops the pipeline here.

04
Distribute
Copy to target Regions and accounts, set launch permissions.

05
Expire
Lifecycle policies deprecate, disable or delete old images.

ON THE EXAM

"Rebuild monthly, or whenever the parent image is patched" is the schedule plus dependency-update trigger. That is the answer to stale AMIs.

AMI lifecycle for operators

- **Copy** to another Region, optionally **re-encrypting** with a different KMS key. An AMI is Regional.
- **Share** with specific accounts, an organisation, or publicly, using launch permissions.
- **Deprecate** hides an AMI from normal listings from a date, without breaking anything that already references it.
- **Deregister** removes it. Anything referencing that AMI ID afterwards **fails to launch**, which breaks Auto Scaling groups.

ON THE EXAM

Deprecate first, deregister much later. Deregistering an AMI still referenced by a launch template turns a scale-out into an outage.

Governing which AMIs may be used

- **Allowed AMIs** is an account-wide, per-Region EC2 setting restricting which AMIs are **discoverable and usable**.
- Criteria: trusted **accounts and owner aliases**, plus **marketplace codes, creation date, deprecation time and AMI name patterns**.
- **Audit mode** applies no restrictions and reports which AMIs **would** be blocked, so you can measure before enforcing.
- It **never restricts AMIs your own account owns**, and it must be configured in every Region, or applied by a declarative policy.

> ON THE EXAM
>
> Audit mode first, then enforce. The same pattern as CloudFormation Hooks' warn mode and predictive scaling's forecast-only mode.

Amazon ECR

- A managed container registry. **Private** repositories per account, and **ECR Public** for images you publish.
- Access is **IAM**: repository policies plus identity policies. There are no registry usernames and passwords.
- **Tag immutability** stops a tag being repointed at different content, so **latest** cannot silently change under you.
- **Lifecycle policies** expire images by age or by count, which is how a repository does not grow forever.

> ON THE EXAM
>
> Immutable tags plus deploying by digest is how you guarantee that what you tested is what you ran.

ECR scanning: basic or enhanced?

BASIC SCANNING

On push, operating system only

- Scans OS packages against a CVE database
- Triggered on push, or run manually
- No additional charge
- A point-in-time result that does not update

ENHANCED SCANNING

Amazon Inspector, continuous

- OS packages and application language packages
- Continuous rescanning as new CVEs are published
- Findings flow into Inspector and Security Hub
- Charged per image scanned

THE TIEBREAKER

Basic tells you an image was clean when you pushed it. Enhanced tells you it is still clean today.

ECR at scale

- **Replication rules** copy images to other Regions and accounts automatically, so a DR Region has the images it needs.
- **Pull through cache** mirrors an upstream registry such as Docker Hub or ECR Public into your own repository on first pull.
- That removes an external dependency at deployment time, and removes upstream rate limits from your critical path.
- Without replication, a disaster recovery Region has **no images**. Module 17 listed this as a classic DR test failure.

> ON THE EXAM
>
> Pull through cache is the answer to "our deployment failed because Docker Hub rate-limited us". It also gives you scanning on borrowed images.

ECS vocabulary

TERM	WHAT IT IS
Cluster	A logical grouping of capacity where tasks run
Task definition	The blueprint: image, CPU, memory, ports, roles, log configuration
Task	One running instantiation of a task definition
Service	Keeps a desired number of tasks running, and registers them with a load balancer
Capacity provider	Where capacity comes from: Fargate, Fargate Spot, or an Auto Scaling group

ON THE EXAM

A task is to a task definition what an EC2 instance is to a launch template. A service is the Auto Scaling group equivalent.

ECS on EC2 or on Fargate?

EC2 LAUNCH TYPE

You own the instances

- Choose instance types, including GPU
- Patch and manage the hosts yourself
- Better density: many tasks per instance
- Cheaper at steady high utilisation

FARGATE

No hosts at all

- Per-task CPU and memory, no instances
- No patching, no capacity planning
- Fargate Spot for interruptible work
- Higher unit price, less operational cost

THE TIEBREAKER

"No infrastructure to manage" means Fargate. Specific instance types, GPUs, or high density means EC2 - or, now, Managed Instances.

ECS Managed Instances

- A newer middle option: **real EC2 instances of any type**, with AWS handling **provisioning, scaling, patching and maintenance**.
- Gives you what Fargate cannot: **GPUs, specific CPU architectures, high network performance**, without owning the hosts.
- ECS launches them using an **infrastructure role** in your account, created for ECS to act on your behalf.
- **SCPs apply to that infrastructure role.** A policy restricting which AMIs may launch will break it unless the ECS AMI accounts are permitted.

> ON THE EXAM
>
> This is a genuine operations trap. A perfectly reasonable security policy silently stops a managed compute option from provisioning anything.

ECS task role or task execution role?

TASK ROLE

What your application can do

- Assumed by the code inside the container
- Read from S3, write to DynamoDB, call an API
- The container's own AWS identity
- Scope it to what the application needs

TASK EXECUTION ROLE

What ECS needs to start it

- Used by the ECS agent, not by your code
- Pull the image from ECR
- Write container logs to CloudWatch Logs
- Fetch secrets from Secrets Manager or SSM

THE TIEBREAKER

"The task will not start and cannot pull the image" is the execution role. "The application gets AccessDenied calling S3" is the task role.

Operating an ECS service

- **Rolling updates** are controlled by minimum and maximum healthy percent, which is how capacity is preserved during a deployment.
- The **deployment circuit breaker** detects a failing deployment and can **roll it back automatically**.
- **Service Auto Scaling** is Application Auto Scaling, from module twelve. Target tracking on CPU, memory or request count.
- **ECS Exec** opens a shell in a running container, using the same channel as Session Manager. No SSH, no bastion.

> ON THE EXAM
>
> ECS Exec is Session Manager for containers, and it is audited the same way. If a question asks how to debug a running container securely, that is the answer.

EKS compute options

OPTION	WHO MANAGES THE NODES
Self-managed nodes	You. Full control, full responsibility, including AMIs and patching
Managed node groups	AWS provisions and updates them; you choose types and scaling ranges
Fargate profiles	No nodes. One pod per micro-VM. No DaemonSets
EKS Auto Mode	AWS entirely: provisioning, patching, and the AMI choice

ON THE EXAM

The trend across ECS and EKS is identical: AWS takes over more of the node lifecycle, and you give up access to the node in exchange.

EKS Auto Mode

- Powered by **Karpenter**, running **Bottlerocket**-based AMIs on **EC2 managed instances**. AWS creates, patches and deletes nodes.
- **No SSH and no SSM access to nodes**, and **no custom AMIs**. That is the trade for AWS owning the operating system.
- **No Windows node support.** Windows workloads stay on managed or self-managed node groups.
- **From 22 April 2026**, new Auto Mode managed instances and their launch templates, volumes and ENIs are **hidden from the EC2 console and describe APIs by default**.

> ON THE EXAM
>
> Auto Mode manages nodes, not pods. You still need HPA or KEDA to decide how many replicas your application runs.

EKS identity: IRSA or Pod Identity?

IRSA

OIDC federation

- An OIDC provider per cluster, trusted by IAM
- Role trust policy references the cluster's OIDC issuer
- Works everywhere, including outside EKS
- More setup, and role trust must be edited per cluster

EKS POD IDENTITY

An agent and an association

- An add-on agent plus a role association
- One simple trust policy, reusable across clusters
- No OIDC provider to create per cluster
- Newer, and the simpler default for new clusters

THE TIEBREAKER

Both give a pod its own IAM role rather than borrowing the node's. That is the point, and it is the same argument as module 3's Pod Identity note.

Watching containers

- **Container Insights** gives cluster, node, pod and task level metrics for both ECS and EKS.
- On ECS: the **awslogs** driver or **FireLens**. On EKS: **Fluent Bit**, from the Observability add-on. Module three covered both.
- Container Insights with **enhanced observability** adds deeper per-container detail, charged per observation.
- **Fargate has no host**, so anything requiring a DaemonSet or a node agent does not apply there.

> ON THE EXAM
>
> The Fargate point recurs. No host means no node-level agent, which changes how logs and metrics are collected.

> An ECS service on Fargate fails to start. Tasks move from PROVISIONING to STOPPED with the message 'CannotPullContainerError: access denied'. The application code has not changed and the image exists in ECR.
>
> The task definition specifies both a task role and a task execution role.

Which role should be investigated?

A The task role, because the container cannot access ECR

B The task execution role, because ECS uses it to pull the image before the container starts

C The ECS cluster's instance profile

D The service-linked role for ECS

◆ PAUSE AND DECIDE

Which role should be investigated?

A The task role

The task role is the application's identity and is only used once the container is running. The failure happens before that.

B The task execution role

Pulling the image, writing logs and fetching secrets all happen under the execution role, before any application code runs.

C Cluster instance profile

There are no instances. Fargate has no hosts, so there is no instance profile to hold permissions.

D ECS service-linked role

Lets ECS manage load balancers and scaling on your behalf. It is not used to pull images.

THE SIGNAL

The stage of failure identifies the role. Anything that happens before your code runs is the execution role's job.

> A security team enables an SCP restricting which AMIs may launch EC2 instances. Only images owned by the company's own accounts are permitted. Shortly afterwards, an EKS Auto Mode cluster stops provisioning new nodes and an ECS Managed Instances capacity provider stops scaling.
>
> No other change was made.

What is the cause?

A. The SCP also applies to the roles those services use, and their AWS-owned AMI accounts are not permitted

B. EKS Auto Mode and ECS Managed Instances require the Allowed AMIs setting to be disabled

C. The clusters need their Kubernetes version upgraded

D. Service-linked roles are exempt from SCPs, so this cannot be the cause

◆ PAUSE AND DECIDE

What is the cause?

A The SCP applies to the service roles

Managed compute launches instances using a role in your account, and that role is subject to your SCPs. The AWS-owned AMI accounts must be permitted.

B Allowed AMIs must be disabled

Allowed AMIs can cause the same symptom and does not have to be disabled. You extend its criteria to include the service AMI accounts.

C Version upgrade needed

Nothing about a version changed. The timing points at the policy, which is the only change described.

D Roles are exempt from SCPs

Confidently wrong. SCPs always apply to actions performed with these infrastructure roles, which is precisely why this breaks.

THE SIGNAL

A reasonable security control silently disables a managed compute option. The fix is to permit the AWS-owned AMI accounts, not to remove the control.

> An organisation requires that every EC2 instance launches from an approved AMI. Developers frequently launch instances from public Marketplace images. The security team wants to understand the impact before enforcing anything, because blocking launches unexpectedly would be disruptive.
>
> They also want compliance reporting on instances already running.

Which combination should the team implement?

A. Allowed AMIs in audit mode first, then enabled, plus the AWS Config rule for instances launched with allowed AMIs

B. An SCP denying ec2:RunInstances outright

C. An IAM policy on each developer restricting AMI IDs

D. EC2 Image Builder pipelines producing approved AMIs

◆ PAUSE AND DECIDE

Which combination should the team implement?

A Audit mode, then enforce, plus the Config rule

Audit mode reports what would be blocked without blocking it, and the Config rule reports existing instances launched from disallowed AMIs.

B Deny RunInstances

Blocks every launch rather than every unapproved launch. It also offers no way to measure impact first.

C Per-developer IAM policies

Enforceable in principle and unmaintainable in practice, requiring every AMI ID listed and updated in every policy.

D Image Builder pipelines

Produces approved AMIs, which is necessary and not sufficient. Nothing stops a developer launching a different one.

THE SIGNAL

Two requirements again: measure before enforcing, and report on what already exists. Audit mode and the Config rule map onto one each.

> A company must be able to run its containerised platform from a second Region within 30 minutes. The EKS cluster definitions are in code and the data layer replicates. During a test, the cluster came up but every pod failed with ImagePullBackOff.
>
> All container images are stored in ECR in the primary Region only.

What should the company implement?

A. ECR cross-Region replication rules, so images exist in the recovery Region

B. ECR pull through cache configured against the primary Region's registry

C. Enhanced scanning on the ECR repositories

D. An ECR lifecycle policy retaining more image versions

◆ PAUSE AND DECIDE

What should the company implement?

A Cross-Region replication

Images are copied to the recovery Region automatically and continuously, so they are already there when the primary Region is not.

B Pull through cache from the primary

The tempting one. A cache populated on first pull depends on the source being reachable, and in a Regional failure it is not.

C Enhanced scanning

Improves vulnerability detection. It does not put an image in another Region.

D Longer image retention

Keeps more versions in the primary Region, which is the Region that is unavailable.

THE SIGNAL

Module 17's point made concrete: DR tests fail on the things that are not data. The cluster was fine and there were no images.

> An operations team runs an EKS cluster in Auto Mode. During an audit they cannot find the worker node EC2 instances in the EC2 console, and describe-instances does not list them. The cluster is running normally and pods are healthy.
>
> The team is concerned that instances are running without visibility.

What is the explanation?

A. Auto Mode uses Fargate, so there are no EC2 instances at all

B. From April 2026, Auto Mode managed instances are hidden from EC2 console views and describe API operations by default

C. The instances are in a different account owned by AWS

D. The audit role lacks ec2:DescribeInstances permission

◆ PAUSE AND DECIDE

What is the explanation?

A **Auto Mode uses Fargate**

It uses EC2 managed instances running Bottlerocket AMIs, provisioned by Karpenter. They are genuinely EC2 instances.

B **Hidden by default since April 2026**

New Auto Mode managed instances and their launch templates, volumes and ENIs are hidden from EC2 console views and describe list operations by default.

C **In an AWS-owned account**

They run in your account and you are billed for them. Ownership is not the reason they are not listed.

D **Missing IAM permission**

A plausible first guess, and it would hide all instances rather than only the Auto Mode ones. Everything else is visible.

THE SIGNAL

Instances that existed before April 2026 remain visible, which is why an estate can show some Auto Mode nodes and not others.

The five things to carry forward

- **Execution role starts the task; task role is the application.** Failure before your code runs is the execution role.
- **Basic scanning is a moment; enhanced scanning is continuous.** Inspector rescans as new CVEs appear.
- **ECR replication puts images in the DR Region.** A pull through cache depends on a source that may be down.
- **Allowed AMIs has an audit mode.** Measure before enforcing, as with Hooks and predictive scaling.
- **Managed compute launches under a role your SCPs govern.** A restrictive AMI policy will silently stop it.

> ON THE EXAM
>
> Next: deployment strategies - rolling, blue/green, canary, immutable, and how to roll back.